

Text Retrieval System - README

Overview

This project implements a **Text Retrieval System** using vector space models with **TF-IDF weighting** and **cosine similarity** for ranking documents. It processes TREC-style documents, builds indexes, and supports queries in three settings:

1. **Title-only.**
2. **Title + Description.**
3. **Title + Narrative.**

Prerequisites

- **Python Version:** Python 3.x.
- **Environment:** Google Colab.
- **Runtime Type:** T4 GPU or CPU.
- **Required Files:**
 - TREC-style FT911 document collection.
 - `stopwords.txt` file.
 - A Python-based Porter stemming algorithm (`porter_stemmer.py`).

Setup Instructions

1. **File Preparation:**
 - a. Upload the required files (`stopwords.txt` and `porter_stemmer.py`) to your Google Colab environment.
 - b. Place the FT911 document collection in Google Drive.
2. **File Conversion:**
 - a. Convert FT911 files into a CSV file (`file_contents.csv`).
 - b. Zip the files for easy handling (`ft911_files.zip`).
3. **Drive Mounting:**
 - a. Mount Google Drive in Colab to access the required data files.

Python Program Process

1. **Document Parsing:**
 - a. Reads each document from the FT911 collection.
 - b. Extracts text between <TEXT> tags using regex.
2. **Tokenization and Filtering:**
 - a. Tokenizes document content into words.
 - b. Removes:
 - i. Stopwords using the stopwords.txt file.
 - ii. Tokens containing numeric characters.
3. **Stemming:**
 - a. Applies the Porter Stemming algorithm using the porter_stemmer.py file.
4. **Index Creation:**
 - a. **Forward Index:**
 - i. Maps each document to its list of terms and their frequencies.
 - b. **Inverted Index:**
 - i. Maps each term to the list of documents containing it.
5. **TF-IDF Representation:**
 - a. Computes **TF-IDF weights** for terms in documents and queries.
6. **Cosine Similarity:**
 - a. Calculates similarity between query and document vectors to rank documents.
7. **Evaluation:**
 - a. Evaluates system performance using precision and recall metrics.

Output Generation

- **Generated Files:**
 - forward_index.txt: Contains the forward index.
 - cosine_similarity_title_desc_vector_and_document_vector.txt: Similarity scores for queries (title + description).
 - cosine_similarity_title_narr_vector_and_document_vector.txt: Similarity scores for queries (title + narrative).

Running the Program

1. Ensure all files are correctly placed and Google Drive is mounted.
2. Run the Python program step-by-step.
3. Verify the generated indexes and similarity scores in the console.

Notes

- **Custom Implementation:**
 - The system does not rely on external NLP libraries like NLTK.
 - The Porter Stemming algorithm is implemented in a custom `porter_stemmer.py` module.
- **File Structure:**
 - Output files follow a structured format for easier evaluation and debugging.
- **Performance Settings:**
 - Query handling supports three configurations:
 - Title-only.
 - Title + Description.
 - Title + Narrative.